Epistemic Assurance: Navigating Undecidability, Abstraction, and Trust in Safety-Critical Systems

Introduction: The Paradox of Provable Incompleteness

The development of safety-critical systems—those employed in aviation, healthcare, and finance—is predicated on a foundational demand for correctness and reliability.[1] In these domains, where the consequences of error can be catastrophic, stakeholders turn to formal methods, a suite of mathematically rigorous techniques designed to prove or disprove the correctness of a system against a formal specification.[2] This pursuit of mathematical certainty, however, confronts a central philosophical tension: it operates in a domain governed by the fundamental limits of computability. The very nature of computation dictates that our knowledge of complex software systems will always be, in some essential way, incomplete.

This report explores this paradox, arguing that "trustworthiness" in software is not a binary property to be proven but rather an *epistemic state* to be managed. The central tool for this management is abstract interpretation, a theory of *sound approximation* of program semantics.[5] Far from being a failure of verification, abstract interpretation represents its most mature and practical form. It is a deliberate, mathematically rigorous methodology for trading absolute completeness for decidability and scalability. This trade-off, however, is not without cost. The process of abstraction inherently involves information loss, creating an "epistemic gap" between what a verification tool can prove about its abstract model and what is actually true of the concrete, running system. This report will analyze the nature of this gap, model its propagation through both automated tools and human cognition, and propose a new framework for its management, culminating in a novel metric for "Epistemic Assurance."

Part I: The Epistemology of Approximation in Formal Methods

Chapter 1: The Inescapable Shadow of Undecidability

The necessity of approximation in formal program verification is not a matter of engineering convenience or a temporary limitation of current technology. It is a direct and unavoidable consequence of the logical structure of computation itself. Any attempt to build a tool that provides definitive, universally correct answers about the behavior of all possible programs is doomed to fail. This fundamental boundary is delineated by two of the most significant results in theoretical computer science: the Halting Problem and Rice's Theorem.

The foundational limit was established by Alan Turing with the **Halting Problem**, which asks whether it is possible to create a general algorithm that can determine, for any given program and input, whether that program will eventually halt or run forever.[7] Turing's proof by contradiction demonstrates that no such algorithm can exist.[9] Any hypothetical "halting checker" could be used to construct a paradoxical program that halts if and only if the checker predicts it will not, leading to a logical impossibility.[7]

This result was powerfully generalized by **Rice's Theorem**, which extends this undecidability to a much broader class of questions.[11] The theorem states that any non-trivial

semantic property of programs is undecidable.[9] A semantic property is one that pertains to the program's behavior or the function it computes (e.g., "does this program terminate for all inputs?", "is the output of this program always zero?"), as opposed to a syntactic property, which pertains to the program's textual structure (e.g., "does the program contain an

if-then-else statement?").[11] Since properties critical to safety and security—such as "this program is free of runtime errors" or "this program is free of security vulnerabilities"—are semantic, Rice's Theorem implies that no general, automated tool can exist to decide them for all programs.[8]

The implications of these theorems are not merely technical but deeply philosophical, echoing Kurt Gödel's incompleteness theorems for mathematics.[12] They establish a fundamental boundary on the extent of formal knowledge obtainable about computational systems. Any formal system powerful enough to describe the behavior of complex software cannot be both complete (able to prove all true statements) and sound (proving only true statements) about that software's behavior. This forces a critical methodological shift. The properties we care most about in safety-critical domains are precisely those that are fundamentally undecidable. Therefore, the pursuit of verification cannot be a quest for absolute, universal proof. Instead, it must become a discipline of managing uncertainty through sound approximation. Abstract

interpretation is the formal, mathematical embodiment of this necessary shift. Our knowledge of a program's

behavior is epistemically distinct from our knowledge of its *text*, and it is this distinction that makes approximation an inescapable feature of formal verification.

Chapter 2: The Lattice of Abstractions as a Map of Knowledge and Ignorance

Abstract interpretation provides the formal framework for navigating the undecidable landscape of program semantics. It does so by systematically replacing an exact, but incomputable, analysis of a program's behavior with a computable, but approximate, one. This process of approximation is not ad-hoc; it is structured by the mathematical theory of lattices and Galois connections, which together create a formal map of what can be known and what must be left uncertain.

The analysis begins with the **concrete semantics**, which represents the complete, true behavior of a program. This can be conceptualized as the set of all possible execution traces a program can produce for all possible inputs.[5] While perfectly precise, this set is generally infinite and thus incomputable.[5] The goal of static analysis is to derive a computable

abstract semantics that *soundly over-approximates* the concrete one.[9] An over-approximation ensures that every possible real behavior is accounted for in the abstract model, even if the model also includes some behaviors that are not actually possible. This guarantees that if the analysis proves a property (e.g., "no division by zero"), that property holds for the real program. The analysis may, however, fail to prove a true property, resulting in a "maybe" or "unknown" answer.[9]

The structure of this approximation is defined by an **abstract domain**, which is a mathematical structure, typically a complete lattice, used to represent abstract properties.[13] A lattice is a partially ordered set where every pair of elements has a least upper bound (join,

$\sqcup$) and a greatest lower bound (meet, $\sqcap$).[15] In abstract interpretation, the partial order

$\sqsubseteq$ represents precision: if a $\sqsubseteq$ b, then a is considered more precise, or contains more information, than b.[9] The lattice has a top element,

$\top$, representing total ignorance (e.g., "a variable can have any value"), and a bottom element, $\bot$, representing a contradiction or an unreachably initial state.[13]

The mathematical bridge between the concrete lattice $(C, \subseteq)$ and the abstract lattice $(A, \sqsubseteq)$ is the **Galois connection**.[16] It consists of two monotonic functions:

- An **abstraction function**, $\alpha: C \to A$, which maps a set of concrete states to the single *most precise* abstract element that soundly represents them.[17] This is the point of deliberate, controlled information loss.
- A **concretization function**, $\gamma: A \to C$, which maps an abstract element back to the full set of concrete states it represents.[17]

These functions are bound by the fundamental property $\alpha(c) \sqsubseteq a \Leftrightarrow c \subseteq \gamma(a)$. A direct consequence is the **soundness condition**: for any concrete element c, it must be that $c \subseteq \gamma(\alpha(c))$.[17] This ensures that the process of abstracting and then concretizing a set of states results in a set that contains, at a minimum, all the original states. No real behaviors are ever lost, they are only potentially obscured by the addition of other, non-existent behaviors in the over-approximation.

Static analysis is then performed by computing a **least fixed point** of abstract transformers (functions that model the effect of program statements on abstract states) over this abstract lattice.[13] For abstract domains with infinite height, such as the interval domain, termination of this computation is not guaranteed. To solve this,

widening ($\triangledown$) and narrowing (Δ) operators are used. Widening accelerates convergence by aggressively over-approximating, jumping up the lattice to find a fixed point quickly, while narrowing refines this approximation downwards to regain some lost precision.[13]

The choice of an abstract domain is therefore not merely a technical decision but an *epistemic commitment*. It pre-defines the universe of questions that can be answered about a program. An interval domain, for example, can represent properties like $x \in$, but it is fundamentally incapable of expressing a property like "x is an even number".[5] A relational domain like convex polyhedra can capture linear relationships between variables (e.g.,

$x \leq y$), but at a significantly higher computational cost than non-relational domains.[5] The lattice of abstractions thus forms a meta-theoretical map of possible verification philosophies. Each domain on this map represents a different trade-off between expressive power, precision, and computational tractability. Designing a static analysis is therefore equivalent to defining an ontology for the verification task—a human-driven declaration of what properties are considered critical and what details can be safely ignored. This choice is laden with assumptions, and it is here that the risk of emergent behavior arises: if a system exhibits a dangerous behavior that is not expressible within the chosen abstract domain, the analysis will be perfectly sound but entirely blind to that specific danger.

Chapter 3: Quantifying Information Loss in the Abstract Domain

While the lattice structure provides a qualitative ordering of precision, a deeper, quantitative understanding of the information loss inherent in abstraction is necessary to assess the epistemic gap it creates. By integrating concepts from information theory and fuzzy logic, it is possible to move beyond simply stating that one abstraction is "more precise" than another and begin to measure the magnitude of the uncertainty introduced.

From an **information theory perspective**, the process of abstraction can be viewed as a form of sound, lossy compression of the program's semantic space.[23] The concrete domain

C can be modeled as a set of all possible program states. The uncertainty associated with this set can be quantified by its Shannon entropy, which measures the average information content or "surprise" of an outcome.[23] When the abstraction function

α maps a set of concrete states c to a single abstract element a, it effectively partitions the state space. The information that is lost is precisely the information required to distinguish between the individual concrete states that were all mapped to a. For example, abstracting the set of concrete integer values {1, 2, 3, 4} to the abstract element pos (positive) loses the $\log_2(4) = 2$ bits of information needed to specify which of the four numbers was the actual value. This loss is a direct measure of the imprecision introduced by the abstraction.

From a **fuzzy logic perspective**, the tripartite output of a static analysis—"definitely true," "definitely false," or "maybe"—can be formalized. Classical logic deals in binary truth values (0 or 1), but the "maybe" result from an abstract interpretation signifies ambiguity.[5] Fuzzy logic provides a mathematical framework for reasoning with degrees of truth, where membership in a set can be a value in the continuous interval `` rather than a binary choice.[24] The "maybe" state can be interpreted as a fuzzy set where the program's true state has a non-zero membership in both the "safe" and "unsafe" categories. The degree of this membership could be derived from the characteristics of the abstract element itself. For instance, if an analysis using the interval domain concludes that a variable

x is in the range [-10, 10] and the unsafe condition is $x < 0$, the degree of "maybe unsafe" could be quantified as being proportional to the size of the intersection of the abstract result with the unsafe region.

These perspectives converge on a formal understanding of precision loss. The soundness condition $c \subseteq \gamma(\alpha(c))$ guarantees that the set of concrete states represented by an abstraction, $\gamma(\alpha(c))$, is a superset of the original set of states c. The information lost corresponds directly to the set of "spurious" states, $\gamma(\alpha(c)) \setminus c$. These are the states that are possible according to the abstract analysis but are not actually reachable in the concrete execution. The size (or measure) of this set, $|\gamma(\alpha(c)) \setminus c|$, provides a direct quantification of the

analysis's imprecision. A more precise abstraction, one that is "higher" in the lattice, will have a smaller concretization $\gamma(a)$, resulting in a smaller set of spurious states and thus a smaller information loss. This provides a formal, information-theoretic grounding for the intuitive notion of precision. It also enables a more direct, quantitative comparison between different abstract domains, moving towards an engineering-driven approach where domains can be selected based on their quantifiable information loss for a given class of problems and safety requirements.

Chapter 4: A Post-Mortem on Abstraction Failure: The Ariane 5 Case Study

The theoretical and philosophical issues surrounding abstraction and information loss are not merely academic. Their real-world consequences were demonstrated catastrophically on June 4, 1996, when the maiden flight of the Ariane 5 rocket, Flight 501, ended in self-destruction just 37 seconds after launch.[26] The failure serves as a canonical case study in how an abstraction, sound in one context, can become fatally unsound when its operational environment changes.

The immediate technical cause of the failure was a software exception in the Inertial Reference System (SRI). A 64-bit floating-point number representing the rocket's horizontal velocity relative to the launch platform was being converted to a 16-bit signed integer. The value of the horizontal velocity on Ariane 5's trajectory was larger than what could be represented in a 16-bit integer (32,767), leading to an operand error.[28] This exception halted the primary SRI processor. The backup SRI, running the same software, failed in the same way moments later. The diagnostic data from the failed SRIs was misinterpreted by the main flight computer as valid flight data, causing it to command extreme nozzle deflections that tore the vehicle apart due to aerodynamic stress.[28]

Viewing this event through the lens of abstract interpretation reveals a deeper, systemic failure. The 16-bit signed integer was, in effect, an **abstract domain** for the physical quantity of horizontal velocity. The concrete domain was the set of real values the velocity could take in flight. The implicit abstraction function was a projection of this real value into the integer range [-32768, 32767]. The software containing this abstraction was reused from the Ariane 4 rocket, a vehicle with a significantly different and less aggressive trajectory.[28] For Ariane 4, the verification and testing were sound; the concrete values of its horizontal velocity never exceeded the limits of the 16-bit integer. The soundness condition

$c \subseteq \gamma(\alpha(c))$ held.

The critical error was the failure to re-validate this abstraction against the new operational

context of Ariane 5. The Ariane 5's trajectory produced a horizontal velocity that exceeded the bounds of the 16-bit abstract domain. At this point, the abstraction was no longer a sound over-approximation; the concrete value lay outside the concretization of the abstract domain. The resulting overflow was not a simple coding bug but a manifestation of this fundamental unsoundness. The problem was compounded by the fact that the alignment function containing this calculation was not even necessary after liftoff for Ariane 5, but it was left running for redundancy reasons, a decision made without a full analysis of its potential impact.[26]

This analysis demonstrates that the root cause was not a failure of software engineering but a failure of **systems engineering**.[26] The software component was, in isolation, "correct" with respect to its original specification for Ariane 4. The trust placed in this reused component was misplaced because it was trust in the code artifact alone, not in the validity of the abstraction it embodied. Abstractions are not timeless mathematical truths; their soundness is contingent upon the physical and operational environment they are intended to model. The Ariane 5 failure fundamentally challenges the notion of simple component reuse in safety-critical systems. It proves that one cannot simply verify a component in isolation and then "trust" it in a new context. One must rigorously re-verify the assumptions of the abstraction at every level of system integration, treating the entire system—code, hardware, and operational environment—as a single, interconnected whole. This was an epistemic failure: a failure to understand the contextual limits of the knowledge encoded within the software.

Part II: A Systems-Theoretic Model of Abstract Verification and Human Interpretation

Chapter 5: Modeling the Verification Ecosystem as a Hierarchical Control Structure

To understand and manage the risks introduced by abstraction, it is necessary to model not just the software or the analysis tool, but the entire socio-technical system in which verification takes place. The Systems-Theoretic Accident Model and Processes (STAMP) provides a powerful framework for this purpose. STAMP reframes the cause of accidents not as a chain of component failures but as the result of inadequate control within a complex system.[30] Using the associated hazard analysis technique, System-Theoretic Process Analysis

(STPA), the verification process can be modeled as a hierarchical control structure, revealing how interactions between the software, the automated tool, and the human analyst can lead to unsafe outcomes.

In this model, the system is composed of feedback control loops, where controllers enforce constraints on a controlled process to prevent hazards. The verification ecosystem can be structured as follows:

- **Level 1: Controlled Process.** This is the **software system under verification**. Its state includes all program variables, memory contents, and execution paths. Hazards are system states that can lead to a loss, such as a buffer overflow, a division by zero, or a data race. The goal of the control system is to enforce safety constraints that prevent the software from entering these hazardous states.
- **Level 2: Automated Controller.** This is the **abstract interpreter** or static analysis tool. It exerts control over the development process by providing diagnostic information.
  - **Control Actions:** The tool issues alerts, such as "Potential null pointer dereference at line X" or "Property Y may be violated." These are the commands intended to guide the human controller's actions.
  - **Process Model:** The tool's "understanding" of the controlled process is its chosen abstract domain and set of abstract transformers. A flaw or imprecision in this model (e.g., an abstraction that is too coarse) directly leads to imprecise or ambiguous control actions (vague "maybe" alerts).
  - **Feedback:** The tool receives feedback by reading the source code of the software system.
- **Level 3: Human Controller.** This is the **software engineer or safety analyst** who interprets the tool's output and makes the final decisions.
  - **Control Actions:** The analyst exerts control by modifying the source code, suppressing an alert (classifying it as a false positive), filing a bug report for a true positive, or ultimately signing off on the code as verified and safe for release.
  - **Process Model:** The analyst's mental model is the most complex and critical component. It includes their understanding of the software's logic, the programming language semantics, the potential system hazards, and, crucially, *the capabilities and limitations of the verification tool itself*. This mental model is the primary locus of failure where cognitive biases can distort interpretation and lead to unsafe control actions.
  - **Feedback:** The analyst receives feedback primarily from the diagnostic alerts issued by the automated controller, supplemented by code documentation, test results, and their own expertise.

Framing the verification process in this way yields a powerful shift in perspective. "False positives" and "unknown" diagnoses are no longer just statistical noise or an inconvenience. They are flawed or ambiguous *control signals* being issued by the automated controller. The overall safety of the system then becomes critically dependent on how the human controller, operating with their own imperfect process model, interprets and acts upon these signals.

This model moves the focus of analysis from just the tool's algorithmic precision to the effectiveness of the entire human-machine control loop. It suggests that improving verification is not solely about building more precise (and computationally expensive) analyzers, but also about improving the interface through which the tool communicates its uncertainty and the processes by which humans are trained to manage it.

Chapter 6: The Human Controller's Flawed Process Model: Cognitive Biases in Diagnostic Interpretation

The STPA model identifies the controller's process model—its internal beliefs about the state of the system it controls—as a primary source of unsafe control actions (UCAs).[30] For the human analyst in the verification loop, this process model is their mental model, and it is systematically vulnerable to cognitive biases. These biases are not random errors but predictable patterns of deviation from rational judgment that can lead an analyst to misinterpret diagnostic alerts and issue unsafe control actions, such as incorrectly dismissing a valid warning or accepting code that contains a latent defect.

Several key cognitive biases are particularly relevant to the task of interpreting static analysis results:

- **Automation Bias:** This is the tendency to over-rely on automated systems, trusting their outputs without sufficient critical scrutiny.[33] In the STPA model, this manifests as a flaw in the analyst's process model: an incorrect belief that "the tool is comprehensive, and an absence of alerts implies an absence of bugs." This can lead to the UCA of signing off on code as verified without performing necessary supplementary manual reviews, especially when the tool's abstract domain was known to be incomplete for certain classes of errors.[35]
- **Confirmation Bias:** This is the tendency to search for, interpret, and recall information in a way that confirms one's pre-existing beliefs.[37] For a developer who believes their code is correct, this bias frames an ambiguous "maybe" alert as likely being a false positive. The flawed process model belief is, "My code is probably correct, so this warning is likely spurious." This leads to the UCA of suppressing a valid but uncertain warning without a rigorous attempt to construct a concrete counterexample that would refute their belief.[39]
- **Anchoring Bias:** This is the tendency to rely too heavily on the first piece of information encountered when making decisions.[39] If an analyst's initial experience with a tool's output on a particular file involves investigating several time-consuming false positives, they may "anchor" on this experience. The flawed belief becomes, "The tool is unreliable for this module." This can lead to the UCA of bulk-suppressing all subsequent alerts of a similar type from that module, potentially ignoring a critical true positive among them.

Furthermore, the well-documented **"Problem Resolution Gap"** can be modeled as a specific flaw in the human controller's process model.[42] When faced with an "unknown" or "maybe" diagnosis, the analyst lacks a clear procedure for how to proceed. The alert provides insufficient information for a direct resolution. This gap in their procedural knowledge often leads to the UCA of ignoring or filtering all ambiguous alerts, which directly compromises system safety by leaving potential defects unexamined.

By translating these psychological concepts into the formal language of systems safety engineering, the human element becomes analyzable within the same rigorous framework as hardware and software components. The following table provides a structured mapping of these cognitive biases to specific flaws in the human controller's process model and the resulting unsafe control actions.

Cognitive Bias	Flaw in Human Controller's Process Model (Incorrect Belief)	Example Unsafe Control Action (UCA)	Causal Scenario Leading to UCA
Automation Bias	"The static analyzer is comprehensive. Absence of alerts implies absence of bugs."	Fails to perform supplementary manual code review on a critical module that passed analysis.	The abstract domain used was not relational and missed a bug involving two variables. The developer, trusting the tool, did not look for this class of error.
Confirmation Bias	"My new algorithm is correct. This 'potential null dereference' warning must be a false positive."	Suppresses a 'maybe' alert without constructing a concrete counterexample to disprove it.	The developer runs a few positive test cases that pass, confirming their belief, and dismisses the tool's warning about a complex edge case.
Anchoring Bias	"The first alert from this tool on this file	Sets a rule to ignore all future	After spending an hour debugging a

	was a false positive. The tool is likely unreliable for this file."	warnings of a certain type from a specific file or module.	spurious warning, the developer decides to preemptively suppress similar warnings to save time.
Availability Heuristic	"The last major bug was a race condition. I should focus only on the concurrency-relate d alerts."	De-prioritizes or ignores a "potential integer overflow" alert in favor of a lower-severity concurrency alert.	A recent, memorable production failure biases the developer's attention and risk assessment towards one class of error over another.

Chapter 7: The Propagation of Epistemic Uncertainty

The systemic risk in a verification process arises not just from the existence of uncertainty but from how it propagates and transforms as it flows through the socio-technical system. The information loss inherent in abstract interpretation creates an initial *epistemic uncertainty*—a lack of knowledge about the system's true state, as distinct from *aleatory uncertainty*, which is inherent randomness.[43] This initial uncertainty is then transmitted to the human analyst, who, influenced by cognitive biases, transforms it before acting. A unified model of this propagation reveals how even a highly precise tool can contribute to an unsafe system if the human interpretation link is flawed.

The propagation can be modeled as a multi-stage process:

1. **Source of Uncertainty:** The abstraction function α is the origin of epistemic uncertainty. As quantified in Chapter 3, this uncertainty, U_formal, is a measure of the imprecision introduced by mapping the concrete state space to the abstract domain. A more precise domain results in a lower initial U_formal.
2. **Transmission of Uncertainty:** The abstract interpreter's output acts as the transmission channel. While "Yes" or "No" answers carry low uncertainty, a "Maybe" or "unknown" result directly transmits U_formal to the human controller. The alert itself is a signal that

the tool's knowledge is incomplete.

3. **Transformation of Uncertainty:** The human controller's mind is the next stage. Their cognitive biases act as a non-linear transfer function, distorting the incoming uncertainty. The objective U_formal is transformed into a subjective U_perceived. For example, confirmation bias might cause an analyst to irrationally discount the probability of a "maybe" alert being a true positive, transforming an objective 50% chance (based on tool history) into a perceived 10% chance.
4. **Output with Residual Uncertainty:** The analyst's final control action—signing off on the code, for instance—carries this miscalibrated residual uncertainty. This final, transformed uncertainty becomes an unmanaged component of the overall systemic risk.

Bayesian Networks offer a formal mechanism for modeling this propagation pathway.[44] A Bayesian Network is a probabilistic graphical model that represents a set of variables and their conditional dependencies. For our verification system, key nodes could include:

- True_State: A binary variable representing the actual presence (1) or absence (0) of a defect.
- Tool_Alert: A variable representing the tool's output (e.g., True_Positive, False_Positive, Unknown). The conditional probability table P(Tool_Alert | True_State) would encode the tool's known precision and recall rates.
- Developer_Belief: A variable representing the analyst's subjective assessment of the alert's validity. The conditional probability P(Developer_Belief | Tool_Alert) would model the influence of cognitive biases. For an unbiased analyst, this would closely track the tool's actual performance. For a biased analyst, it would be skewed.
- Developer_Action: A variable for the final decision (e.g., Fix_Code, Suppress_Alert).

This model makes it clear that the final systemic risk is not a simple sum of individual component risks but the product of propagated and transformed uncertainties. A highly precise analysis tool (low U_formal) can still lead to a high-risk system if it is operated by a human with strong cognitive biases (a highly distorting transfer function). Conversely, a less precise tool might be part of a safer overall system if its human operators are acutely aware of its limitations and have established, bias-mitigating protocols for handling ambiguous diagnostics. This provides a compelling argument for a socio-technical approach to assurance, where investing in processes that improve human interpretation—such as mandatory peer review, checklists, and training in statistical reasoning—may yield a higher return on investment for safety than purchasing a marginally more precise analysis algorithm. The problem shifts from "how do we eliminate all false positives?" to the more tractable and realistic question, "how do we build a system that is resilient to ambiguous information?".

Part III: Towards a Metric for Epistemic Assurance

Chapter 8: Defining the Epistemic Assurance Metric (EAM)

To effectively manage the socio-technical risks inherent in abstract verification, a new type of metric is required. Traditional software quality metrics—such as defect density, test coverage, or cyclomatic complexity—measure intrinsic properties of the code artifact itself.[46] While useful, they fail to capture the quality or trustworthiness of the

verification process that was applied to that code. They can tell us how complex the code is, but not how much confidence we should have that its complexities have been adequately analyzed.

To address this gap, this report proposes the **Epistemic Assurance Metric (EAM)**, a composite metric designed to quantify the holistic, socio-technical confidence in a verification outcome. The EAM is defined as a function of two key components: the formal confidence derived from the analysis tool and the calibrated trust in the human interpretation process.

The proposed formulation is:

EAM=Cformal×Tcalibrated
Where:

- **C_formal (Formal Confidence):** This is a score in the range `` representing the precision and soundness of the formal analysis itself. It is derived directly from the quantitative measures of information loss developed in Chapter 3. A value of 1 would represent a (hypothetical) sound and complete analysis with zero information loss, while a value approaching 0 would represent an analysis that is so abstract as to be useless (e.g., one that always returns $\top$, or "unknown"). For a given piece of code, an analysis using a highly precise abstract domain like Polyhedra would yield a higher C_formal score than one using a less precise domain like Signs.
- **T_calibrated (Calibrated Human Trust):** This is a score in the range `` representing the quality and rationality of the human interpretation and decision-making process. This is critically not a measure of an analyst's subjective "trust" in the tool, but rather a measure of how well-calibrated their judgment is to the tool's actual capabilities. A score of 1 implies a perfectly rational analyst who is immune to cognitive biases and accurately weighs the evidence presented by the tool. A score approaching 0 represents a process where decisions are made irrationally, for instance, by ignoring all tool outputs.

The multiplicative nature of the EAM is its most crucial feature. It formally models the idea that

assurance is a chain of dependencies where the weakest link determines the overall strength. Even if a formal analysis is perfect (C_formal = 1), if the human process is completely untrustworthy (T_calibrated = 0, e.g., an analyst who automatically suppresses all alerts without review), the overall epistemic assurance is zero. Conversely, a perfectly rational and diligent human process (T_calibrated = 1) cannot create assurance out of a useless, imprecise analysis (C_formal = 0). An additive relationship would fail to capture this critical dependency.

The EAM provides a quantitative framework for making strategic investments in safety. It allows an organization to model the return on investment (ROI) of different interventions. For example, a manager could compare the projected increase in EAM from purchasing a more expensive, precise analysis tool (which would increase C_formal) against the projected increase from implementing a mandatory peer-review process for all alert dispositions (which would increase T_calibrated). This shifts the focus from a purely technology-centric view of verification to a balanced, socio-technical one.

Chapter 9: A Methodology for Measuring Calibrated Human Trust

The practical utility of the Epistemic Assurance Metric hinges on the ability to operationalize its components. While C_formal can be derived from the properties of the abstract domain, measuring T_calibrated requires a novel methodology that goes beyond simple self-reported trust to assess the objective quality of human judgment.

Research in Human-Computer Interaction (HCI) has produced numerous validated scales for measuring user trust in automated systems, such as the Trust in Automation Scale (TIAS) and the Trust of Automated Systems Test (TOAST).[49] These scales typically use questionnaires to gauge user perceptions across dimensions like system performance, reliability, integrity, and the user's overall confidence.[37] However, a high level of self-reported trust is not always desirable. If trust exceeds the actual capability of the system, it can lead to complacency and misuse, a phenomenon known as automation bias.[33] Therefore, the goal is not to measure trust itself, but to measure

calibrated trust—trust that is appropriately aligned with the system's objective performance.

To measure T_calibrated, the following three-step methodology is proposed:

1. **Baseline Trust Measurement:** An analyst or team of analysts is first given a standard, validated trust questionnaire (e.g., a version of the TIAS adapted for static analysis tools). This provides a baseline measure of their subjective, self-reported trust in the tool, yielding a score T_self_reported. The items would probe confidence, perceived reliability, and dependability (e.g., "I am confident in the system," "The system is reliable").[49]
2. **Bias Measurement:** The same analysts are then subjected to a controlled experiment

designed to measure their susceptibility to relevant cognitive biases. This would involve presenting them with a curated set of static analysis alerts for a sample codebase. This set would include a mix of clear true positives, clear false positives, and, most importantly, a significant number of ambiguous "maybe" alerts that require deep investigation. The analyst's task is to disposition each alert (e.g., classify as "True Bug," "False Positive," or "Needs More Info"). Their performance is scored based on objective criteria, such as the accuracy in identifying known true positives and the tendency to prematurely dismiss ambiguous alerts that are, in fact, true positives. This process yields a Bias_Score in the range ``, where 0 represents perfectly rational, unbiased performance and 1 represents maximally biased performance (e.g., dismissing all ambiguous alerts).

3. Calibration Function: The final T_calibrated score is calculated by adjusting the self-reported trust score based on the objectively measured bias. A simple but effective calibration function would be:

   $$T_{calibrated} = T_{self_reported} \times (1 - Bias_Score)$$

   This function penalizes the subjective trust score in proportion to the analyst's demonstrated bias. An analyst who reports high trust but performs poorly in the bias experiment (high Bias_Score) will receive a low T_calibrated score, reflecting that their high confidence is misplaced and poorly calibrated.

This methodology reframes trust from a subjective feeling into an objective, performance-based competency. T_calibrated becomes a metric of an analyst's *skill* in forming justified beliefs and making rational decisions based on the uncertain, incomplete information provided by an abstract interpretation tool. This provides a pathway for professional development, training, and even certification for analysts in safety-critical domains. Organizations could use this metric to identify individuals who require further training in critical thinking and bias mitigation, and to assign their most highly calibrated analysts to the verification of their most critical systems.

Chapter 10: Accounting for the Unknown: Emergent Behaviors and the EAM

The most profound challenge to any verification methodology is the problem of "unknown unknowns"—dangerous system behaviors that were not anticipated by designers and are therefore not present in any specification. These **emergent behaviors** arise from the complex, non-linear interactions of system components and are not reducible to the properties of those components in isolation.[30] They represent the fundamental limit of any

verification approach based on checking a system against a predefined set of properties.

Abstract interpretation, by its very nature, is designed to find violations of properties that can be expressed within a predefined abstract domain. It is a powerful tool for finding known classes of bugs (e.g., null dereferences, overflows, data races) but is structurally incapable of detecting a hazardous emergent property that its chosen domain cannot represent. This is the ultimate epistemic blind spot.

While formal methods are limited here, other engineering disciplines have been developed to probe for emergence. **Resilience testing** and **chaos engineering**, for example, are empirical techniques that intentionally inject faults and stress into a system to observe its behavior under adverse conditions.[56] The goal is not to verify conformance to a specification, but to discover unexpected weaknesses and failure modes, thereby building confidence in the system's ability to handle unforeseen events.[56]

A comprehensive assurance metric must acknowledge the residual risk posed by emergence. The EAM can be extended to incorporate this epistemic humility by introducing a penalty term, the "Emergence Risk Factor" (R_emerge). The final metric becomes:

$$EAM_{final} = EAM \times (1 - R_{emerge})$$

The R_emerge factor would be a score in the range ``, where 0 indicates low potential for dangerous emergence and 1 indicates high potential. This score would not be derived from formal analysis but from a structured assessment of system characteristics known to correlate with emergent behavior. These could include:

- **System Complexity:** Using static analysis metrics like cyclomatic complexity, coupling (afferent and efferent), and lines of code to quantify the structural complexity that can hide unforeseen interactions.[47]
- **Concurrency and Distribution:** The degree of parallelism and the number of interacting distributed components, as these are primary sources of non-linear interactions.
- **Non-Determinism:** The use of components with non-deterministic behavior, such as machine learning models or complex scheduling algorithms.
- **Operational Novelty:** The degree of difference between the system's intended operational environment and the environments in which it was tested and verified. A highly novel environment increases the likelihood of encountering unforeseen conditions.

The purpose of this modified metric, EAM_final, is not to prove the absence of dangerous emergent behavior, which is impossible. Its purpose is to quantify our epistemic humility. The R_emerge factor is a formal, explicit admission of the limits of our foresight. A high R_emerge score signals that even if the formal analysis is highly precise (C_formal is high) and the human analysts are perfectly rational (T_calibrated is high), there remains a significant residual risk stemming from the system's inherent complexity that cannot be eliminated by formal analysis alone. This provides a quantitative argument for mandating supplementary assurance activities. A low EAM_final score, driven by a high R_emerge, would justify the need

for extensive resilience testing, red teaming, or the design of robust runtime monitoring and fail-safe mechanisms to manage the irreducible risk of emergence.

Chapter 11: A Hypothetical Application of the EAM

To illustrate the practical utility of the Epistemic Assurance Metric, it is instructive to apply it in a post-hoc analysis of the Ariane 5 Flight 501 verification process. This demonstrates how the EAM could have quantitatively flagged the process as carrying unacceptable epistemic risk, even when traditional testing and review failed to do so.

- **Calculating C_formal (Formal Confidence):** The critical flaw resided in the conversion of a 64-bit float to a 16-bit signed integer. The 16-bit integer served as the abstract domain for horizontal velocity. This is an extremely coarse abstraction that discards a vast amount of information about the potential range of the physical value. Compared to using a 64-bit floating-point representation throughout (which would be a much more precise abstract domain), the information loss is immense. Consequently, the C_formal score for this part of the verification would have been very low, reflecting the high potential for the abstraction to become unsound.
- **Calculating T_calibrated (Calibrated Human Trust):** The decision to reuse the Ariane 4 SRI component without a full re-evaluation of its operational constraints in the new Ariane 5 context is a textbook example of multiple cognitive biases at play. It demonstrates **status quo bias** (continuing to use what was used before), **automation bias** (placing undue trust in a component previously deemed "proven" or "reliable"), and **confirmation bias** (assuming the component would work and not seeking evidence to refute this assumption). A controlled bias measurement experiment would likely have revealed a high susceptibility to these biases among the decision-makers. This would result in a high Bias_Score and, consequently, a very low T_calibrated score, indicating that the trust placed in the component was dangerously miscalibrated.
- **Calculating R_emerge (Emergence Risk Factor):** The Ariane 5's flight trajectory was significantly different and more aggressive than that of the Ariane 4. This represents a high degree of **operational novelty**. The interaction between the new flight dynamics (an environmental factor) and the old software component's internal limitations (the 16-bit abstraction) produced the catastrophic failure—a classic emergent behavior. A structured assessment of this novelty would have resulted in a non-trivial, and likely high, R_emerge score.
- Final Result: The final metric would be the product of three low values:

  $$EAMfinal=(\text{low Cformal})\times(\text{low Tcalibrated})\times(1-\text{high Remerge})$$

  The resulting EAM_final score would be extremely low. This quantitative result would have

served as a powerful, unambiguous warning. It would have flagged the verification process itself as carrying significant epistemic risk, providing a clear and compelling justification to management for delaying the launch and demanding a more rigorous, system-level re-verification of all assumptions, particularly those embedded in reused software components.

Conclusion: Beyond Correctness to Calibrated Confidence

The quest for certainty in safety-critical software is a noble but ultimately impossible goal, bounded by the fundamental limits of computation. This report has traced the journey from the theoretical undecidability of program properties to the practical necessity of sound approximation through abstract interpretation. This process, while mathematically rigorous, introduces an epistemic gap—a quantifiable loss of information that creates uncertainty. This uncertainty propagates from the automated analysis tool to the human engineer, whose own cognitive biases can distort its meaning, leading to flawed judgments and potentially catastrophic failures, as exemplified by the Ariane 5 disaster.

Trustworthiness, therefore, is not an intrinsic property of code to be proven, but an emergent property of the entire socio-technical system that produces and verifies it. A myopic focus on improving only the algorithmic precision of analysis tools is insufficient. The human-machine interface—the point where formal uncertainty is translated into human belief and action—is an equally critical component of the safety case.

The proposed Epistemic Assurance Metric (EAM) provides a new language and a quantitative framework for managing this complex reality. By composing formal confidence (C_formal), calibrated human trust (T_calibrated), and a penalty for emergence risk (R_emerge), the EAM offers a holistic measure of the assurance process itself. It moves the goalposts from the Sisyphean search for absolute "correctness" to the practical and achievable goal of *calibrated confidence*. It forces engineers, managers, and regulators to be explicit about what is known, what is unknown, and how much we can justifiably trust our own judgment in the face of irreducible ambiguity. In the high-stakes world of safety-critical systems, acknowledging the limits of our knowledge is the first and most crucial step toward genuine assurance.

Works cited

1. Formal Methods and Logic - Penn Computer & Information Science Highlights, accessed on August 29, 2025, https://highlights.cis.upenn.edu/formal-methods-and-logic/
2. Formal Verification Methods - MATLAB & Simulink - MathWorks, accessed on

August 29, 2025, https://www.mathworks.com/discovery/formal-verification.html
3. Formal verification - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Formal_verification
4. Verification and formal methods: Academic Centre of Cyber Security Excellence, The University of Melbourne - School of Computing and Information Systems, accessed on August 29, 2025, https://cis.unimelb.edu.au/research/cyber-security-excellence/research/verify-formal-methods
5. Abstract interpretation - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Abstract_interpretation
6. Abstract Interpretation: Theory and Practice - NYU Computer Science, accessed on August 29, 2025, https://cs.nyu.edu/~pcousot/publications.www/Cousot-SPIN02.pdf
7. Halting problem - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Halting_problem
8. The Philosophy Behind Formal Verification - Number Analytics, accessed on August 29, 2025, https://www.numberanalytics.com/blog/philosophy-behind-formal-verification
9. Introduction to Abstract Interpretation - Bruno Blanchet, accessed on August 29, 2025, https://bblanche.gitlabpages.inria.fr/absint.pdf
10. What are the fundamental limits of computation behind the Halting Problem and Rice's Theorem? : r/compsci - Reddit, accessed on August 29, 2025, https://www.reddit.com/r/compsci/comments/1lyyzc9/what_are_the_fundamental_limits_of_computation/
11. Rice's theorem - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Rice%27s_theorem
12. Philosophical Foundations of Formal Verification - Number Analytics, accessed on August 29, 2025, https://www.numberanalytics.com/blog/philosophical-foundations-of-formal-verification
13. Abstract Interpretation - UT Computer Science, accessed on August 29, 2025, https://www.cs.utexas.edu/~isil/cs389L/AI-6up.pdf
14. The Ultimate Guide to Abstract Interpretation - Number Analytics, accessed on August 29, 2025, https://www.numberanalytics.com/blog/ultimate-guide-abstract-interpretation
15. Chapter 5. Lattices, closure operators, and Galois connections. - UC Berkeley math, accessed on August 29, 2025, https://math.berkeley.edu/~gbergman/245/2.4/Ch.5.pdf
16. Galois connection - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Galois_connection
17. A Galois Connection Calculus for Abstract Interpretation*, accessed on August 29, 2025, https://cs.nyu.edu/~pcousot/publications.www/CousotCousot-POPL14-ACM-p2-3-2014.pdf
18. On Constructivity of Galois Connections, accessed on August 29, 2025,

https://www.math.unipd.it/~ranzato/papers/vmcai18.pdf
19. Lecturecise 10 Galois Connection and Abstract Interpretation - LARA, accessed on August 29, 2025, https://lara.epfl.ch/w/_media/sav17:lecturecise10.pdf
20. Introduction to Abstract Interpretation - CiteSeerX, accessed on August 29, 2025, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=cd9fcedc2213a3a85bc9b74b5d5db2308eb0e075
21. A Personal Historical Perspective on Abstract Interpretation - NYU Computer Science, accessed on August 29, 2025, https://cs.nyu.edu/~pcousot/publications.www/Cousot-FSP-2024.pdf
22. Abstract Interpretation - PKC - Obsidian Publish, accessed on August 29, 2025, https://publish.obsidian.md/pkc/Hub/Theory/Sciences/Computer+Science/Abstract+Interpretation
23. Information theory - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Information_theory
24. Fuzzy Logic Concepts, Developments and Implementation - MDPI, accessed on August 29, 2025, https://www.mdpi.com/2078-2489/15/10/656
25. Fuzzy Logic: Definition, Meaning, Examples, and History - Investopedia, accessed on August 29, 2025, https://www.investopedia.com/terms/f/fuzzy-logic.asp
26. (PDF) The Ariane 5 Flight 501 Failure - A Case Study in System ..., accessed on August 29, 2025, https://www.researchgate.net/publication/2637332_The_Ariane_5_Flight_501_Failure_-_A_Case_Study_in_System_Engineering_for_Computing_Systems
27. Why is Formal Methods Necessary?, accessed on August 29, 2025, https://shemesh.larc.nasa.gov/fm/fm-why-new.html
28. The Ariane 5 Failure: How a Huge Disaster Paved the Way for Better Coding - D-RisQ, accessed on August 29, 2025, https://www.drisq.com/the-ariane-5-failure-how-a-huge-disaster-paved-the-way-for-better-coding
29. An analysis of the Ariane 5 flight 501 failure-a system engineering perspective., accessed on August 29, 2025, https://www.researchgate.net/publication/220882405_An_analysis_of_the_Ariane_5_flight_501_failure-a_system_engineering_perspective
30. STPA: A Systems Approach to Process Hazard Analysis — GATE ..., accessed on August 29, 2025, https://www.gate.energy/the-brainery/stpa
31. Introduction to STAMP, STPA and CAST - UL Solutions, accessed on August 29, 2025, https://www.ul.com/sis/blog/introduction-to-stamp-stpa-and-cast
32. A System-Theoretic Approach to Risk Analysis - DSpace@MIT, accessed on August 29, 2025, https://dspace.mit.edu/bitstream/handle/1721.1/147729/yoo-samyoo-sm-sdm-2021-thesis.pdf?sequence=1&isAllowed=y
33. Automation bias - Wikipedia, accessed on August 29, 2025, https://en.wikipedia.org/wiki/Automation_bias
34. Automation bias: a systematic review of frequency, effect mediators, and mitigators - PMC, accessed on August 29, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC3240751/

35. Automation Bias: Can Algorithms Perpetuate Discrimination and Inequality? - WeblineIndia, accessed on August 29, 2025, https://www.weblineindia.com/blog/automation-bias/
36. AI Safety and Automation Bias | CSET, accessed on August 29, 2025, https://cset.georgetown.edu/wp-content/uploads/CSET-AI-Safety-and-Automation-Bias.pdf
37. (PDF) Confirmation Bias in Software Development and Testing: An ..., accessed on August 29, 2025, https://www.researchgate.net/publication/235430372_Confirmation_Bias_in_Software_Development_and_Testing_An_Analysis_of_the_Effects_of_Company_Size_Experience_and_Reasoning_Skills
38. What is Confirmation Bias? — updated 2025 | IxDF - The Interaction Design Foundation, accessed on August 29, 2025, https://www.interaction-design.org/literature/topics/confirmation-bias
39. Cognitive bias: Impacts on software development - BoatyardX, accessed on August 29, 2025, https://boatyardx.com/cognitive-bias-impacts-on-software-development/
40. Confirmation Bias in QA: Unveiling the Hidden Traps - MagicPod, accessed on August 29, 2025, https://blog.magicpod.com/confirmation-bias-in-qa-unveiling-the-hidden-traps
41. Countering Cognitive Biases in Software Development and User Experience Design, accessed on August 29, 2025, https://madeintandem.com/blog/countering-cognitive-biases-software-development-user-experience-design/
42. (PDF) Challenges with Responding to Static Analysis Tool Alerts, accessed on August 29, 2025, https://www.researchgate.net/publication/331792299_Challenges_with_Responding_to_Static_Analysis_Tool_Alerts
43. Uncertainty Propagation through Software Dependability Models - SciSpace, accessed on August 29, 2025, https://scispace.com/pdf/uncertainty-propagation-through-software-dependability-27y138eizz.pdf
44. Bayesian network based software reliability prediction with an ..., accessed on August 29, 2025, https://www.researchgate.net/publication/223365621_Bayesian_network_based_software_reliability_prediction_with_an_operational_profile
45. Bayesian networks for system reliability reassessment - CORE, accessed on August 29, 2025, https://core.ac.uk/download/pdf/191098533.pdf
46. Essential Metrics for the QA Process - BrowserStack, accessed on August 29, 2025, https://www.browserstack.com/guide/essential-qa-metrics
47. 7 Metrics to Evaluate your code quality using static analysis - DEV Community, accessed on August 29, 2025, https://dev.to/bahaanoah/7-metrics-to-evaluate-your-code-quality-using-static-analysis-4hpl
48. Top Five Code Metrics to Help You Test Better - TestRail, accessed on August 29,

2025, https://www.testrail.com/blog/code-metrics-to-test-better/
49. Measuring trust in artificial intelligence: validation of an established ..., accessed on August 29, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC12098057/
50. Measuring Human-Computer Trust INTRODUCTION - CiteSeerX, accessed on August 29, 2025, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=b8eda9593fbcb63b7ced1866853d9622737533a2
51. Foundations for an Empirically Determined Scale of Trust in Automated Systems, accessed on August 29, 2025, https://www.researchgate.net/publication/247502831_Foundations_for_an_Empirically_Determined_Scale_of_Trust_in_Automated_Systems
52. Measuring User Trust in Systems - Test Science, accessed on August 29, 2025, https://testscience.org/measuring-user-trust-in-systems/
53. Measuring trust in artificial intelligence: validation of an established scale and its short form, accessed on August 29, 2025, https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1582880/full
54. Challenges in Detecting Emergent Behavior in System Testing, accessed on August 29, 2025, https://www.gaudisite.nl/INCOSE2021_KjeldaasEtAL.pdf
55. Systems of Systems & Emergent Behavior -Important Lessons, accessed on August 29, 2025, https://vmcse.com/2025/05/22/systems-of-systems-emergent-behavior/
56. What is Resilience Testing: The Ultimate Guide | Speedscale, accessed on August 29, 2025, https://speedscale.com/blog/resilience-testing/
57. System Resilience Part 6: Verification and Validation - Software Engineering Institute, accessed on August 29, 2025, https://www.sei.cmu.edu/blog/system-resilience-part-6-verification-and-validation/